Exact SAT Encoding for the N-Queens Problem: An Empirical Comparison with MIP and CP

Truong Dinh Duc[a]

[a]Faculty of Information Technology, VNU University of Engineering and Technology, Hanoi, 100000, Vietnam

ARTICLE INFO

Keywords:
N-Queens
SAT encoding
cardinality constraints
AMO
PySAT
MIP
constraint programming
empirical evaluation

ABSTRACT

The N-Queens problem asks for a placement of n non-attacking queens on an $n \times n$ board and is a classical testbed for combinatorial solving. We present an exact SAT-based approach that encodes N-Queens through at-most-one (AMO) and at-least-one (ALO) cardinality constraints, and we study how the choice of cardinality encoding interacts with the choice of SAT solver. We implement five encodings from scratch (pairwise, binary, commander, product, sequential) and seven encodings available in PySAT, and we evaluate eight SAT solvers on instances with $n = 8$ up to $n = 80$ (three repetitions, 600 s time limit). We compare the SAT approach against seven exact baselines implemented with Gurobi, CPLEX (MIP and CP), OR-Tools CP-SAT, OR-Tools MIP (SCIP), and PuLP/CBC. The experiments show a strong encoding–solver interaction: the pairwise encoding is the most robust and is dominant for weaker solvers, whereas the binary encoding is fastest for strong solvers. MiniSat and MiniCard are the most effective SAT solvers overall, while Glucose times out on the harder encodings. Among the exact baselines, Gurobi is by far the fastest, with an almost flat runtime, and CPLEX MIP is second; the best SAT configuration is competitive with CP-SAT and CPLEX CP but clearly slower than the two commercial MIP solvers on this feasibility-oriented problem.

1. Introduction

The N-Queens problem consists in placing n queens on an $n \times n$ chessboard so that no two queens attack each other, i.e., no two queens share a row, a column, or a diagonal. It is a well-known constraint satisfaction problem that has been studied for decades both for its combinatorial structure and as a benchmark for exact and heuristic algorithms [2]. Although the existence of a solution is known for every $n \geq 4$, finding a satisfying placement and comparing the efficiency of different exact technologies remain useful case studies.

Modern Satisfiability (SAT) solvers, based on conflict-driven clause learning (CDCL) [13], have become an effective engine for a wide range of combinatorial problems. N-Queens can be encoded into conjunctive normal form (CNF) by representing each board cell by a Boolean variable and by expressing the attacking constraints and the "one queen per row" condition as cardinality constraints. The efficiency of the resulting CNF depends heavily on the encoding of the at-most-one (AMO) and at-least-one (ALO) constraints [12, 15, 16].

In this work we ask how the choice of cardinality encoding interacts with the choice of SAT solver, and how SAT compares with other exact paradigms such as Mixed-Integer Programming (MIP) and Constraint Programming (CP) on N-Queens. Our contributions are as follows.

- We implement five AMO encodings from scratch (pairwise, binary, commander, product, and the sequential-counter encoding of Sinz) and use seven encodings provided by the PySAT library, within a single reusable Python package.
- We benchmark eight SAT solvers (MiniSat, MiniCard, CaDiCaL 1.5.3 and 1.9.5, MergeSAT, Lingeling, Glucose 3 and 4) on instances from $n = 8$ to $n = 80$ with three repetitions and a 600 s time limit.
- We compare SAT with seven exact baselines built with Gurobi, CPLEX (MIP and CP), OR-Tools CP-SAT, OR-Tools MIP (SCIP backend), and PuLP (CBC backend).
- We report and discuss a strong encoding–solver interaction, a solver-specific preprocessing artefact, and the gap between SAT and commercial MIP/CP solvers on this problem.

ORCID(s):

The rest of the paper is organized as follows. Section 2 defines the problem, reviews cardinality encodings, and discusses related work. Section 3 presents the CNF model, the encodings, the solver harness, and the baselines. Section 4 reports the computational experiments and discusses the results. Section 5 concludes.

2. Background

2.1. Problem definition and CNF model

Let $x_{r,c}$ be a Boolean variable that is true if and only if a queen is placed in row r and column c, for $0 \le r, c < n$, so that n^2 variables are used. A valid placement satisfies the following constraints:

$$\sum_{c=0}^{n-1} x_{r,c} = 1 \qquad \forall r \quad \text{(exactly one per row)}, \tag{1}$$

$$\sum_{r=0}^{n-1} x_{r,c} \le 1 \qquad \forall c \quad \text{(at most one per column)}, \tag{2}$$

$$\sum_{(r,c)\in D} x_{r,c} \le 1 \qquad \forall \text{diagonal } D, \tag{3}$$

where a diagonal is a maximal set of cells with constant $r - c$ (main diagonals) or constant $r + c$ (anti-diagonals). Because every row contains exactly one queen, there are exactly n queens; consequently the column and diagonal constraints only need the "at most one" direction. The exactly-one condition of a row is encoded as the conjunction of an ALO constraint and an AMO constraint, while columns and diagonals require only an AMO constraint.

2.2. Cardinality encodings

An AMO constraint over literals $\ell_1, \dots, \ell_m$ states that at most one literal is true, and an ALO constraint states that at least one literal is true. We consider the following AMO encodings, where m is the number of literals.

- **Pairwise.** One clause $(\neg\ell_i \vee \neg\ell_j)$ for every unordered pair $\{i, j\}$: $\binom{m}{2}$ clauses, no auxiliary variables. Propagation is strong but the CNF is large.
- **Binary.** Each literal is assigned a distinct $\lceil \log_2 m \rceil$-bit code and implications force the code; this uses $m\lceil \log_2 m \rceil$ clauses and $\lceil \log_2 m \rceil$ auxiliary variables.
- **Commander.** Literals are partitioned into groups of size at most g; each group gets a commander variable, AMO is enforced inside each group and among the commanders, recursively. This reduces the clause count to $O(m)$ at the cost of additional variables.
- **Product.** Literals are arranged on a $p \times q$ grid ($pq \ge m$); implications relate each literal to a row and a column variable, and AMO is enforced on the row and column variables.
- **Sequential (Sinz).** A chain of $m - 1$ auxiliary variables encodes "the number of true literals seen so far is at most one", using $3m - 4$ clauses and $m - 1$ auxiliary variables; this is the standard sequential-counter encoding [16].

In addition, PySAT [11] provides the *bitwise*, *ladder*, *sorting-network*, *cardinality-network*, and *totalizer* encodings, which we include for completeness. The clause and variable counts of each encoding for $n = 40$ are reported in Table 1.

2.3. Related work

N-Queens has a long history in combinatorics [2], and its decision version (N-Queens Completion) has been shown to be NP-complete [8], which motivates generic exact solvers. On the SAT side, cardinality encodings are a central topic in the Handbook of Satisfiability [4, 15], with the sequential counter of Sinz [16], the pairwise and commander constructions [12], and the binary and product variants [7] being the most widely used. PySAT [11] exposes these encodings through a uniform interface, and modern CDCL solvers such as MiniSat [5], Glucose [1], and CaDiCaL/Kissat [3] provide strong search engines. On the optimization side, MIP and CP solvers such as Gurobi [9], CPLEX [10], OR-Tools [14], and CBC [6] are the standard exact baselines. Our study combines these strands by systematically comparing cardinality encodings, SAT solvers, and MIP/CP baselines on a single N-Queens model.

Table 1
CNF size of each encoding for n = 40.

encoding	#vars	#clauses
pairwise	1600	103520
binary	2776	35584
commander	4668	18006
product	4042	18516
sequential	7762	18292

3. Method

3.1. Model construction

We build the CNF with rows encoded as exactly-one (ALO + AMO) and columns and diagonals encoded as AMO, as described in Section 2. The original variables occupy the identifiers $1, \ldots, n^2$ and auxiliary variables are allocated afterwards. Two diagonal encodings are provided: a *naive* mode that adds the pairwise clause $(\neg x_{r_1,c_1} \vee \neg x_{r_2,c_2})$ for every pair of cells on a common diagonal (a quadratic expansion per diagonal), and an *optimal* mode that applies the selected AMO encoding once per diagonal, which keeps the number of diagonals linear and reuses the same encoder.

3.2. Encodings and solvers

All five hand-written encodings and the seven PySAT encodings share one interface, so that the same model can be instantiated with any encoding. Solving is performed through PySAT; each solve is executed in an isolated subprocess so that a crashing or hanging solver is recorded as `crash` or `timeout` rather than aborting the experiment. For every run we record the CNF build time, the solver time, the number of variables and clauses, and the solver statistics (decisions, propagations, and conflicts).

3.3. MIP/CP baselines

The same N-Queens model is expressed with binary variables $a_{r,c}$ subject to exactly-one per row, at-most-one per column, and at-most-one per diagonal, using the following engines: Gurobi [9], CPLEX MIP and CPLEX CP [10], OR-Tools CP-SAT [14], OR-Tools MIP with the SCIP backend, and PuLP with the CBC backend [6]. All baselines minimise a constant objective (the problem is a feasibility problem) and are run under the same time limit as the SAT solvers.

4. Computational experiments

4.1. Experimental setup

All experiments were run on a Windows machine with an 11th-generation Intel processor, using Python 3.11 and PySAT 1.8/1.9. We used the SAT solvers MiniSat (`minisat22`), MiniCard (`minicard`), CaDiCaL 1.5.3 and 1.9.5, MergeSAT 3, Lingeling, and Glucose 3 and 4. Instances range from $n = 8$ to $n = 80$ in steps of two. Each configuration was repeated three times and the median is reported; the time limit is 600 s per run. Two PySAT solvers available on some platforms (`cryptosat`, `minisatgh`) were not shipped in the distribution used here, and `kissat404` reproducibly crashes on these instances; these solvers are therefore excluded. All reported instances are satisfiable, so we measure the time to *find* a model rather than to prove unsatisfiability.

4.2. CNF structure

Table 1 shows the CNF size of each encoding for $n = 40$. The pairwise encoding uses the fewest variables (the n^2 original variables only) but the most clauses, whereas the sequential encoding uses the most variables and relatively few clauses. The commander and product encodings need fewer clauses than binary but introduce more auxiliary variables.

4.3. SAT solvers and encodings

Table 2 ranks the SAT solvers by their median solving time over all encodings for $n \geq 40$, together with the number of timeouts. MiniSat and MiniCard are the fastest and most robust, followed closely by the two CaDiCaL versions; Glucose 3 and 4 are substantially slower and are the only solvers that time out.

Table 2
SAT solver ranking over $n \geq 40$ (median time across encodings) and number of timeouts.

solver	median time (s)	timeouts
minisat22	0.421	0
minicard	0.428	0
cadical153	1.358	0
cadical195	1.518	0
mergesat3	1.871	0
lingeling	2.168	0
glucose4	19.373	13
glucose3	27.532	80

Table 3
Median SAT solving time (s) for $n \geq 40$ by encoding and solver (– = all runs timed out).

encoding	minisat22	minicard	cadical153	cadical195	mergesat3	lingeling	glucose4	glucose3
pairwise	0.599	0.621	0.945	0.949	0.591	0.583	0.775	0.810
binary	0.192	0.205	1.056	0.949	0.819	1.796	16.887	48.854
commander	0.598	0.611	1.039	1.299	5.328	20.168	47.784	203.359
product	0.461	0.467	2.120	2.080	1.871	13.433	20.317	291.456
sequential	0.482	0.476	2.048	2.594	4.121	1.552	25.639	30.234

Table 4
Median solving time (s) of the MIP/CP baselines.

n	gurobi	cplex_mp	cpsat	cplex_cp	mip_scip	mip_cbc	pulp_cbc
16	0.004	0.011	0.029	0.058	0.089	2.287	3.438
24	0.003	0.012	0.037	0.089	0.087	0.081	0.168
32	0.004	0.021	0.076	0.103	0.133	0.131	0.320
40	0.004	0.029	0.108	0.134	2.603	0.367	0.368
48	0.004	0.033	0.157	0.217	0.558	0.852	0.604
56	0.005	0.041	0.201	0.264	8.414	1.086	1.265
64	0.006	0.058	0.262	0.337	15.124	1.545	1.109
72	0.006	0.058	0.351	0.424	8.391	1.645	1.787
80	0.005	0.098	0.474	0.609	3.170	1.284	2.749

Table 3 and Figure 1 show the median time as a function of both the encoding and the solver. They reveal a strong *encoding–solver interaction*: no single encoding dominates. The pairwise encoding is uniformly fast (between 0.6 and 0.8 s for all solvers), whereas the binary encoding is the best choice for MiniSat and MiniCard (0.19–0.21 s) but is dramatically worse for Glucose (up to 48.9 s). The product and commander encodings are the weakest for Glucose, exceeding 200 s.

Figure 2 shows the time as a function of n for each encoding with one curve per solver. The curves for pairwise are tightly clustered, indicating robustness across solvers, whereas the other encodings diverge strongly as n grows, with the Glucose curves rising several orders of magnitude above the rest.

4.4. Comparison with MIP and CP

Table 4 and Figure 3 report the MIP/CP baselines. Gurobi is by far the fastest, with an almost constant runtime of about 0.005 s regardless of n; CPLEX MIP is second at roughly 0.1 s for $n = 80$. CP-SAT and CPLEX CP grow more noticeably, while the SCIP- and CBC-based MIP baselines are slower and noisier.

Figure 4 compares the best SAT configuration (binary encoding with MiniSat) against all baselines. For $n \geq 60$ the best SAT approach is on the order of 0.3–0.9 s, comparable to CP-SAT and CPLEX CP and clearly faster than the CBC/SCIP baselines, but slower than CPLEX MIP and far slower than Gurobi. The gap to Gurobi widens with n.

4.5. Discussion

Three observations stand out. First, the encoding–solver interaction means that encoding quality cannot be judged in isolation: pairwise wins for propagation-weak solvers because its many clauses give strong propagation, while binary wins for strong CDCL solvers because it keeps the formula small. This is consistent with the classical view that the best encoding depends on the solver [15]. Second, we observed a solver-specific artefact: for the small pairwise instance $n = 8$, CaDiCaL reports zero decisions, propagations, and conflicts because it solves the instance entirely during preprocessing, without entering its search loop; the counters therefore measure search effort only, not total work. This confirms that propagation-based statistics should be used with care when comparing heterogeneous solvers. Finally, the commercial MIP solver Gurobi dominates all other exact approaches on this feasibility-oriented problem, which highlights that, for tiny feasibility instances, a tuned MIP presolve and heuristic can outperform a generic SAT workflow even when the latter is carefully encoded. The advantage of SAT is its modularity: the same model runs with many solvers and encodings, and the best configuration remains competitive with CP-SAT and CPLEX CP.

5. Conclusion

We presented an exact SAT-based approach for N-Queens, implemented with five hand-written and seven PySAT cardinality encodings and eight SAT solvers, and compared it with seven MIP/CP baselines. Our experiments show a strong encoding–solver interaction, identify MiniSat and MiniCard as the most effective solvers and pairwise and binary as the best encodings depending on the solver, and confirm the strength of commercial MIP on this problem. Future work includes extending the study to the optimisation variant of N-Queens (maximising the number of queens on an $n \times m$ board), to harder unsatisfiable instances, and to additional SAT solvers once the crashing and missing solvers become available.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used a large language model to assist with code development, text drafting, and language revision. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: International Joint Conference on Artificial Intelligence (IJCAI 2009), pp. 399–404.

[2] Bell, J., Stevens, B., 2009. A survey of known results and research areas for n-queens. Discrete Mathematics 309, 1–31. doi:10.1016/j.disc.2007.12.043.

[3] Biere, A., Fazekas, K., Fleury, M., Heisinger, M., 2020. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT competition 2020, in: SAT Competition 2020 — Solver and Benchmark Descriptions.

[4] Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), 2021. Handbook of Satisfiability. volume 336 of *Frontiers in Artificial Intelligence and Applications*. 2 ed., IOS Press. doi:10.3233/FAIA336.

[5] Eén, N., Sörensson, N., 2003. An extensible SAT-solver, in: Theory and Applications of Satisfiability Testing (SAT 2003), Springer. pp. 502–518. doi:10.1007/978-3-540-24605-3_37.

[6] Forrest, J., Lougee-Heimer, R., 2005. CBC User Guide. COIN-OR Foundation.

[7] Frisch, A.M., Giannaros, P.A., 2010. SAT Encodings of the At-Most-k Constraint: Some Initial Observations. Technical Report. University of York.

[8] Gent, I.P., Jefferson, C., Nightingale, P., 2017. Complexity of n-queens completion. Journal of Artificial Intelligence Research 59, 815–848. doi:10.1613/jair.5512.

[9] Gurobi Optimization, LLC, 2024. Gurobi Optimizer Reference Manual. Gurobi Optimization, LLC. URL: https://www.gurobi.com.

[10] IBM Corporation, 2022. IBM ILOG CPLEX Optimization Studio. IBM Corporation. URL: https://www.ibm.com/products/ilog-cplex-optimization-studio.

[11] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing (SAT 2018), Springer. pp. 428–437. doi:10.1007/978-3-319-94144-8_26.

[12] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects, in: Workshop on Constraints in Formal Verification (CFV 2007).

[13] Marques-Silva, J.P., Sakallah, K.A., 1999. GRASP: A search algorithm for propositional satisfiability. IEEE Transactions on Computers 48, 506–521. doi:10.1109/12.769433.

[14] Perron, L., Furnon, V., 2023. OR-Tools. Google. URL: https://developers.google.com/optimization.

[15] Prestwich, S.D., 2009. CNF encodings, in: Handbook of Satisfiability. IOS Press, pp. 75–97. doi:10.3233/978-1-58603-929-5-75.

[16] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming (CP 2005), Springer. pp. 827–831. doi:10.1007/11564751_73.

Research Data

The source code, benchmark data, and the scripts that reproduce all figures and tables are available at:

https://github.com/Duc-2705/n-Queens

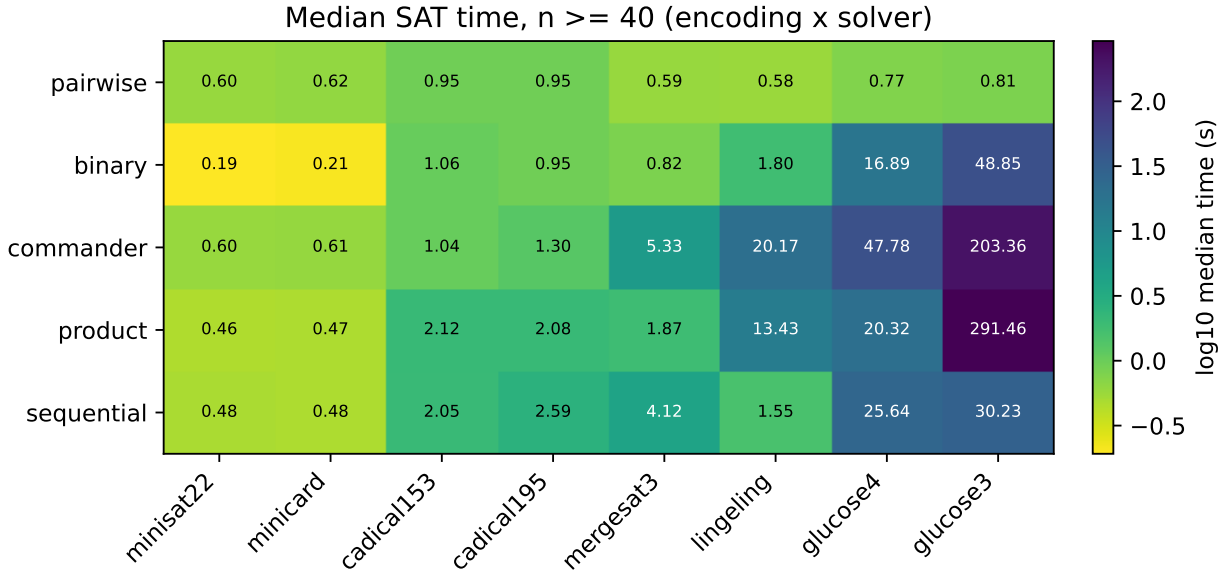


Figure 1: Median SAT solving time ($n \geq 40$) for each encoding–solver pair; darker is slower. Missing cells correspond to configurations in which all runs timed out.

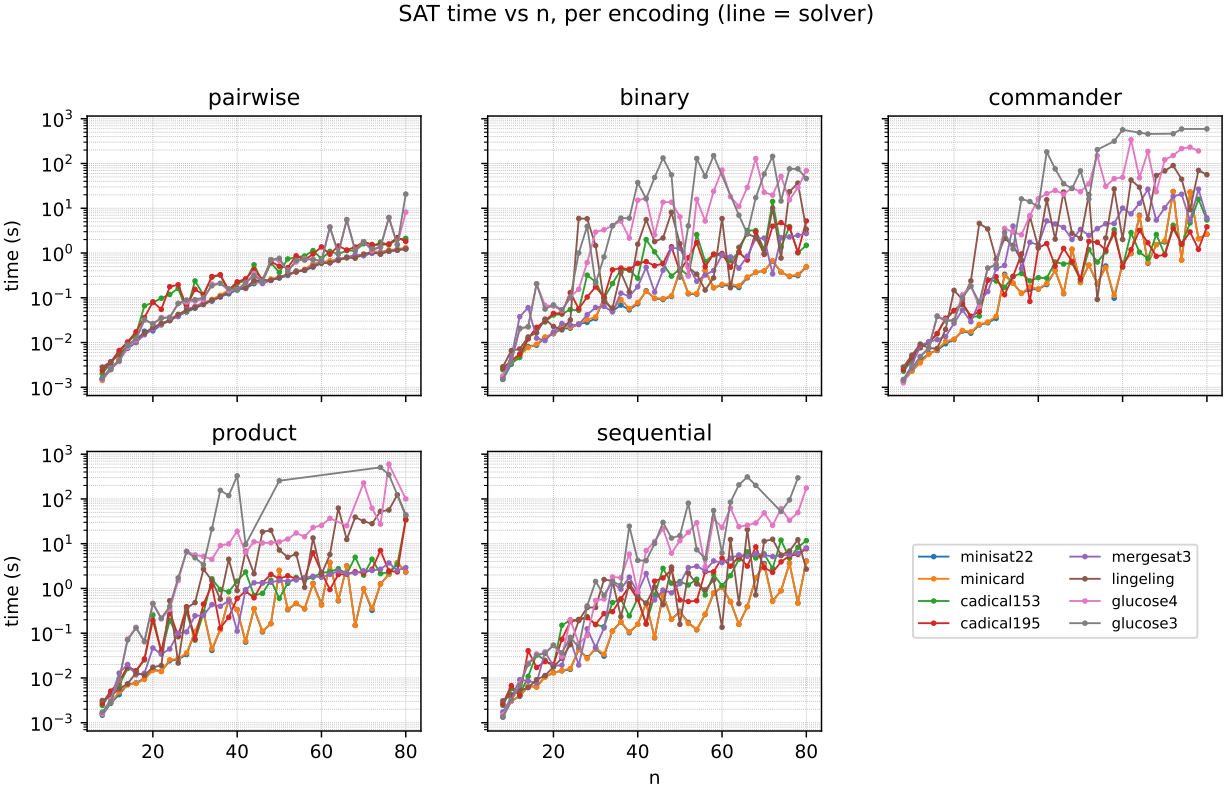


Figure 2: SAT solving time versus n for each encoding (line = solver). The pairwise panel is the most uniform; the remaining panels separate strong and weak solvers.

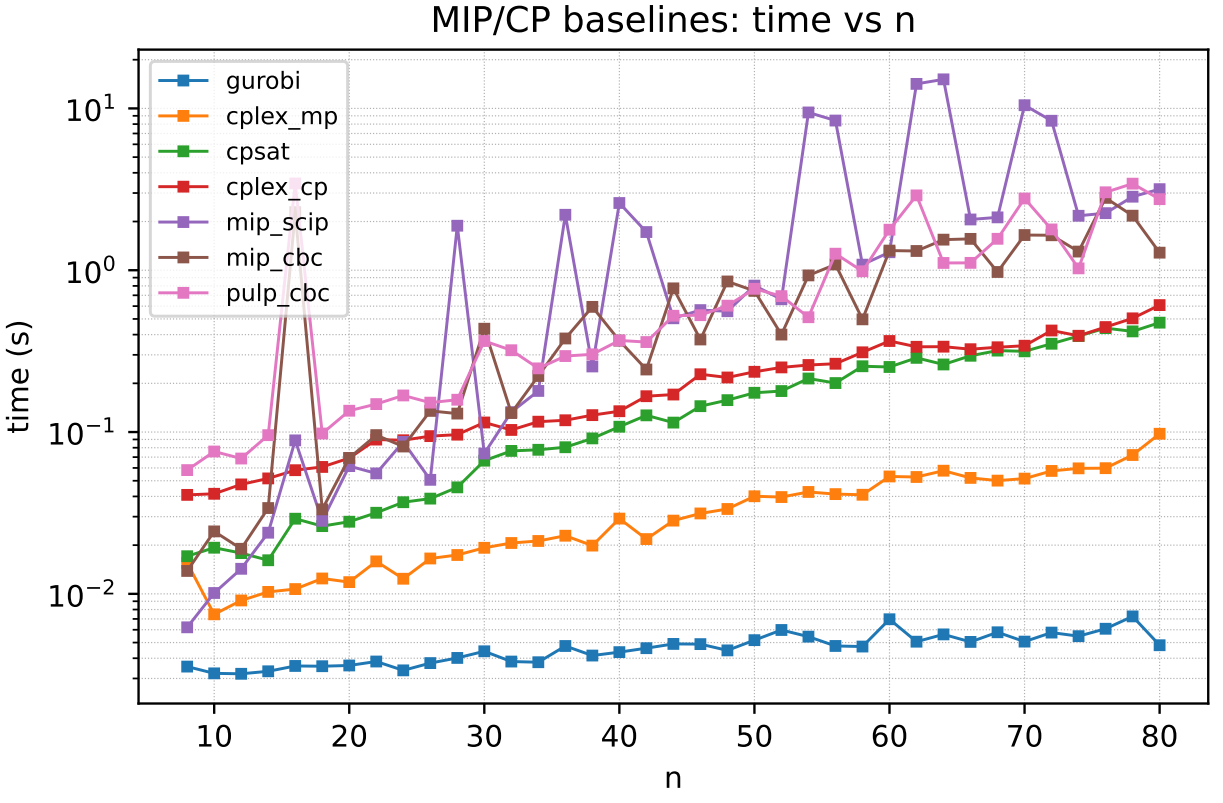


Figure 3: Median solving time of the MIP/CP baselines as a function of n.

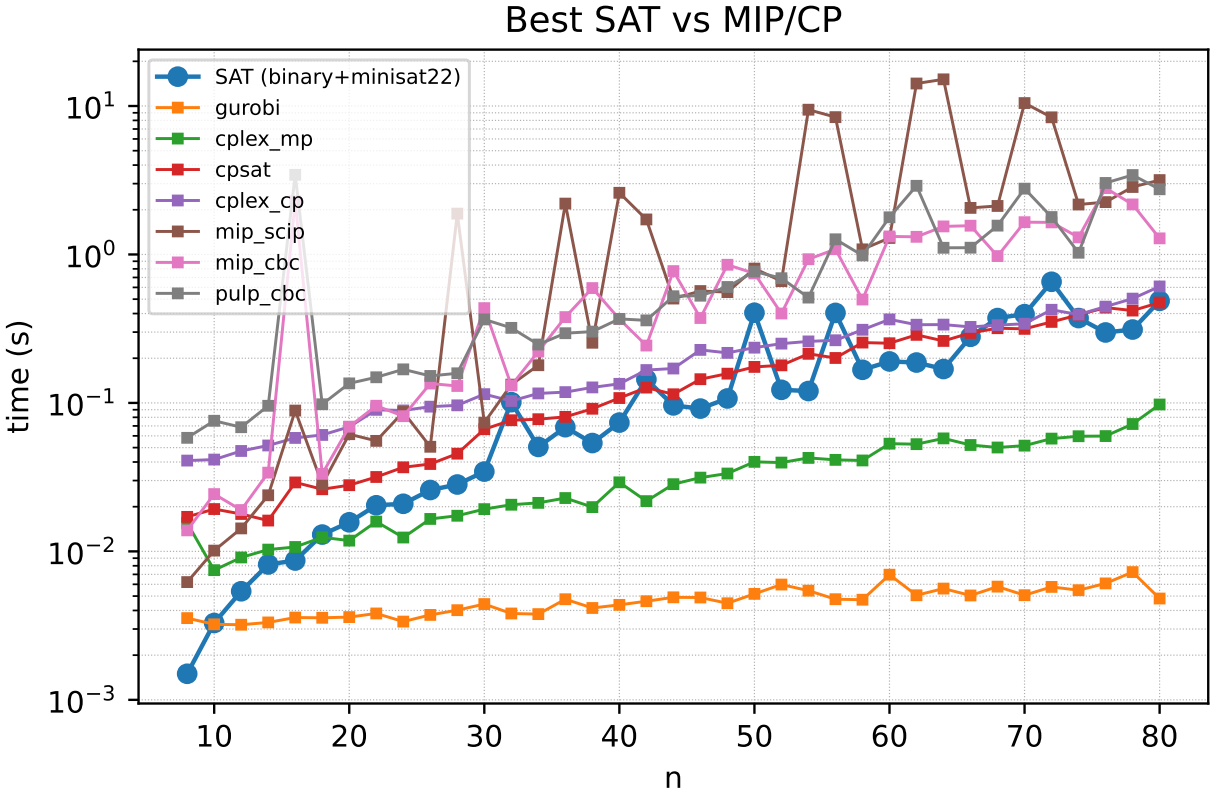


Figure 4: Best SAT configuration (binary encoding with MiniSat) versus the MIP/CP baselines.